

LAPORAN PRAKTIKUM
TEKNOLOGI IOT 2



Disusun Oleh :
Muhamad Ramadhani (25782045)

Dosen Pembimbing :
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..
Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address berikut:

```
#include <ESP8266WiFi.h>
```

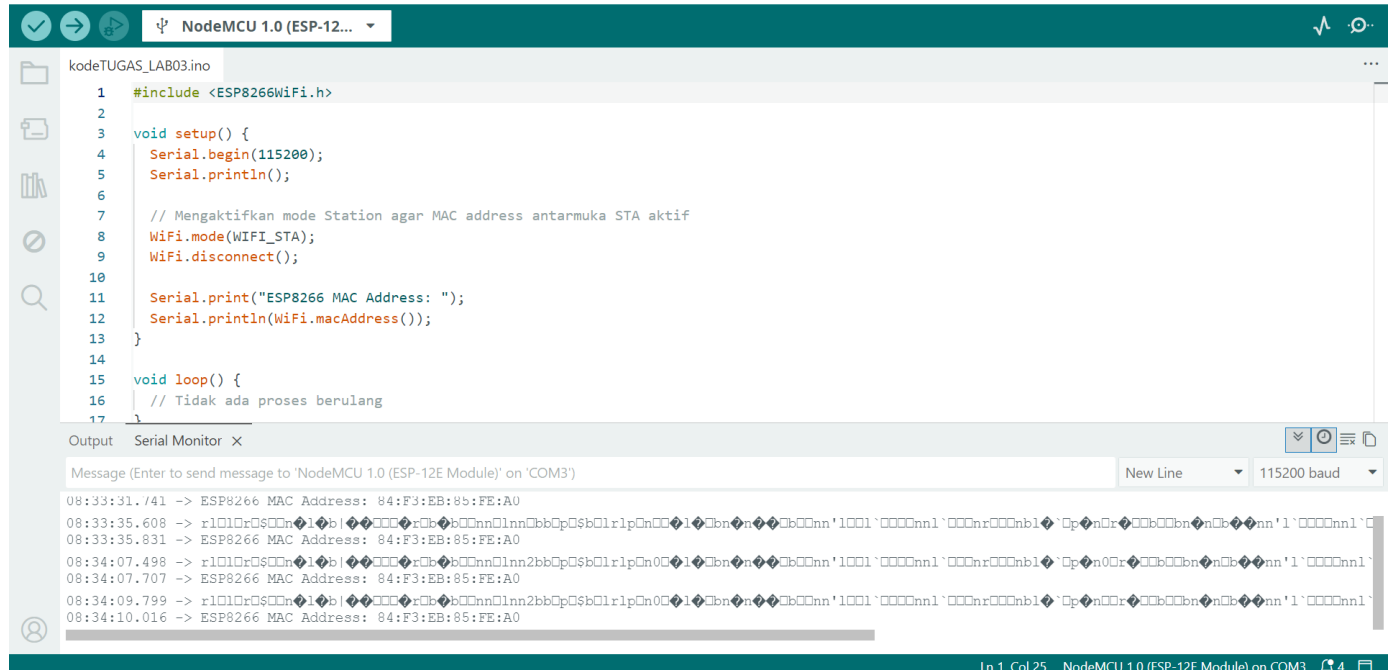
```
void setup() {  
  Serial.begin(115200);  
  Serial.println();  
  
  // Mengaktifkan mode Station agar MAC address antarmuka STA aktif  
  WiFi.mode(WIFI_STA);  
  WiFi.disconnect();  
  
  Serial.print("ESP8266 MAC Address: ");  
  Serial.println(WiFi.macAddress());  
}
```

```
void loop() {  
  // Tidak ada proses berulang  
}
```

3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90).
6. Beri label fisik pada *board* (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

Penjelasan Singkat Kode:

- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.



The screenshot shows the Arduino IDE interface with the file `kodeTUGAS_LAB03.ino` open. The code is as follows:

```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

The Serial Monitor is open, showing the output of the code. The first line of output is:

```
08:33:31.741 -> ESP8266 MAC Address: 84:F3:EB:85:FE:A0
```

Subsequent lines show the same output, indicating that the MAC address has been successfully retrieved and printed.

Disini saya berhasil mendapatkan Alamat macc address saya dengan nomor 84:F3:EB:85:FE:A0

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller)

```
#include <ESP8266WiFi.h>
```

```
#include <espnw.h>
```

```
// Struktur data penerima (wajib identik dengan sisi pengirim)
```

```
typedef struct struct_pesan {
```

```
    int perintahId;
```

```
    int nilaiParameter;
```

```
} struct_pesan;
```

```
struct_pesan paketTerima;
```

```
// Callback otomatis saat antenna mendeteksi paket masuk
```

```
void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
```

```
    char macStr[18];
```

```
    snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
```

```
             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
```

```
// Salin buffer memori radio langsung ke struktur data variabel
```

```
memcpy(&paketTerima, incomingData, sizeof(paketTerima));
```

```
Serial.print("Paket masuk dari: ");
```

```
Serial.println(macStr);
```

```
Serial.print("Ukuran Data: ");
```

```
Serial.print(len);
```

```
Serial.println(" byte");
```

```
Serial.print("Perintah ID: ");
```

```
Serial.println(paketTerima.perintahId);
```

```
Serial.print("Nilai Parameter: ");
```

```
Serial.println(paketTerima.nilaiParameter);
```

```
Serial.println("-----");
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    WiFi.mode(WIFI_STA);
```

```
    WiFi.disconnect();
```

```
if (esp_now_init() != 0) {
```

```
    Serial.println("Inisialisasi ESP-NOW Gagal!");
```

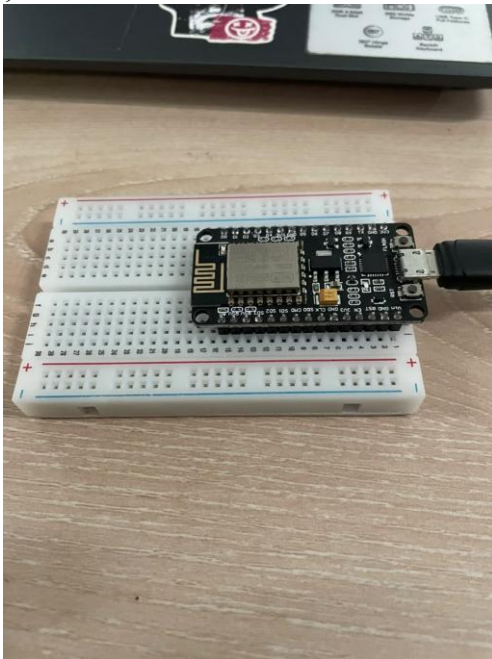
```
    return;
```

```
}
```

```
// Tetapkan peran board sebagai Slave
esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
esp_now_register_recv_cb(OnDataRecv);

Serial.println("Receiver ESP-NOW Menunggu Paket...");
}
```

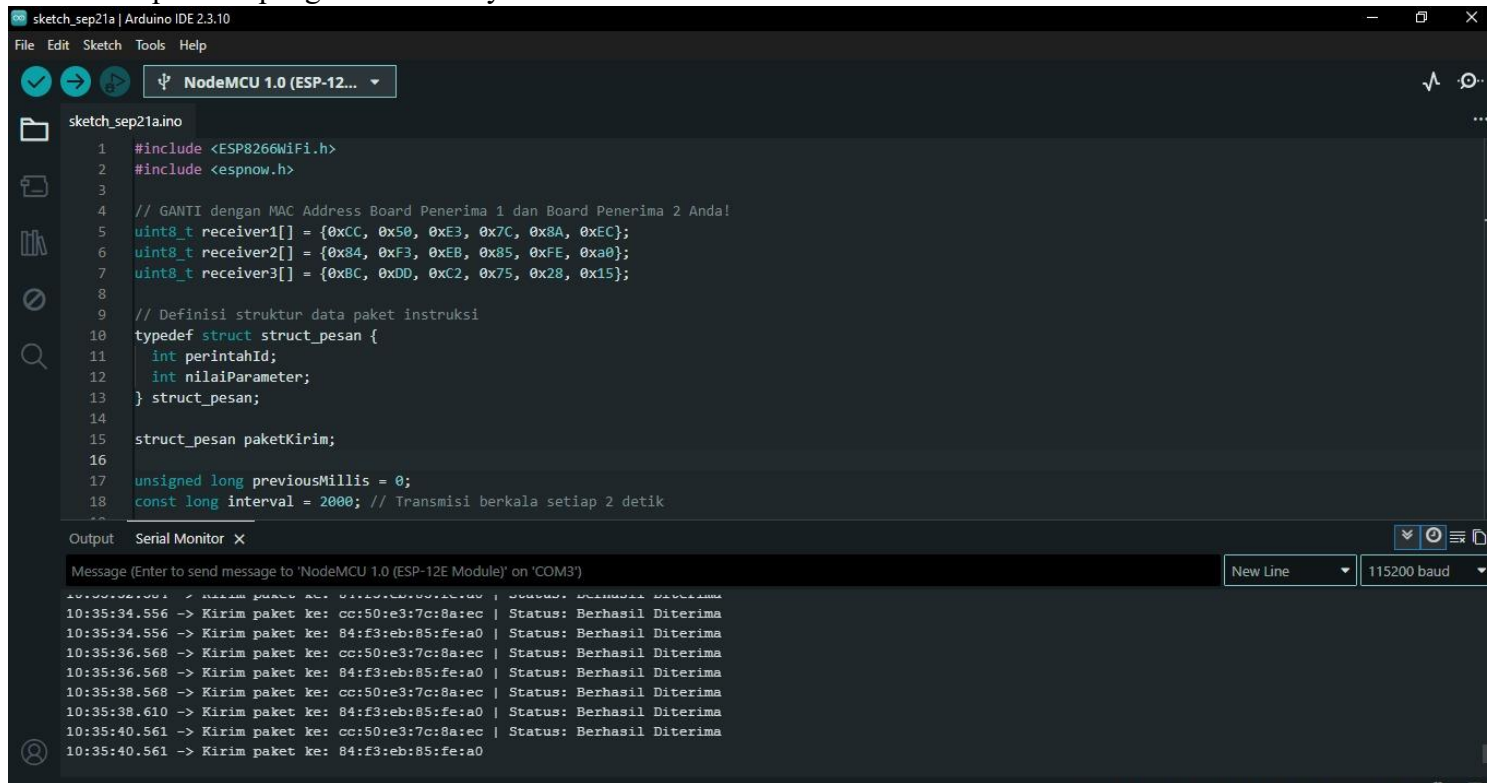
```
void loop() {
    // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
}
```



```
kodeTUGAS_LAB03.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahId;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17 }
18
19 Output Serial Monitor X
20 Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
21
22 08:50:34.844 -> Nilai Parameter: 14
23 08:50:34.844 -> -----
24 08:50:36.810 -> Paket masuk dari: 68:c6:3a:cd:12:d7
25 08:50:36.810 -> Ukuran Data: 8 byte
26 08:50:36.810 -> Perintah ID: 101
27 08:50:36.858 -> Nilai Parameter: 22
28 08:50:36.858 -> -----
29
30 indexing: 36/88
31 Ln 52, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3 4
```

Ini saya berperan sebagai penerima dari macc adres 84:F3:EB:85:FE:A0 yang sudah berhasil masuk outputnya ke serial monitor

Dan ini output dari pengirim teman saya



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading, along with a dropdown menu for the board type, currently set to 'NodeMCU 1.0 (ESP-12E Module)'. The sketch file is named 'sketch_sep21a.ino'. The code in the sketch defines MAC addresses for three receivers, defines a packet structure, and sets up a serial transmission interval of 2000ms. The Serial Monitor at the bottom shows the output of the sketch, displaying the MAC addresses and the status of the transmission.

```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5  uint8_t receiver1[] = {0xCC, 0x50, 0xE3, 0x7C, 0x8A, 0xEC};
6  uint8_t receiver2[] = {0x84, 0xF3, 0xEB, 0x85, 0xFE, 0xA0};
7  uint8_t receiver3[] = {0xBC, 0xDD, 0xC2, 0x75, 0x28, 0x15};
8
9  // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

```
10:35:32.561 -> Kirim paket ke: 01:13:02:00:1C:1D | Status: Berhasil Diterima
10:35:34.556 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:35:34.556 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:35:36.568 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:35:36.568 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:35:38.568 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:35:38.610 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:35:40.561 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:35:40.561 -> Kirim paket ke: 84:f3:eb:85:fe:a0
```

5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik **Board Penerima Pusat (Hub)** pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.
4. Unggah kode program berikut:

```
#include <ESP8266WiFi.h>
```

```
#include <espnow.h>
```

```
// GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
```

```
uint8_t hubMacAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
```

```
// Identitas unik node (Ubah angka ini untuk node yang berbeda!)
```

```
#define BOARD_ID 1
```

```
typedef struct struct_sensor {
```

```
    int id;
```

```
    float pembacaan1;
```

```
    float pembacaan2;
```

```
} struct_sensor;
```

```
struct_sensor dataKirim;
```

```
unsigned long prevTime = 0;
```

```
const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
```

```
void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

```
    Serial.print("Pengiriman Node #");
```

```
    Serial.print(BOARD_ID);
```

```
    Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    WiFi.mode(WIFI_STA);
```

```
    WiFi.disconnect();
```

```
    if (esp_now_init() != 0) {
```

```
        Serial.println("Inisialisasi ESP-NOW gagal");
```

```
        return;
```

```
    }
```

```

esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
esp_now_register_send_cb(OnDataSent);

// Daftarkan hub penerima tunggal
esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
    unsigned long now = millis();
    if (now - prevTime >= sendInterval) {
        prevTime = now;

        dataKirim.id = BOARD_ID;
        dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
        dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %

        esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
    }
}

```

Untuk foto rangkaian masi sama dengan lab 2 tadi

The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, uploading, and other functions. The main editor window displays the code for 'kodeTUGAS_LAB03.ino'. The code includes headers for ESP8266WiFi.h and espnow.h, defines a MAC address for the hub, and sets a BOARD_ID of 2. It defines a struct for sensor data and declares a global variable dataKirim. The loop function sends data to the hub every 1000 milliseconds. The serial monitor at the bottom shows the output of the program, displaying the time and the status of the data transmission.

```

kodeTUGAS_LAB03.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5  uint8_t hubMacAddress[] = {0x68, 0xC6, 0x3A, 0xCD, 0x12, 0xD7};
6
7  // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8  #define BOARD_ID 2
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 void setup() {
19     Serial.begin(115200);
20     esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
21     esp_now_register_send_cb(OnDataSent);
22     // Daftarkan hub penerima tunggal
23     esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
24 }
25
26 void loop() {
27     unsigned long now = millis();
28     if (now - prevTime >= sendInterval) {
29         prevTime = now;
30
31         dataKirim.id = BOARD_ID;
32         dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
33         dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %
34
35         esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
36     }
37 }

```

Output Serial Monitor X

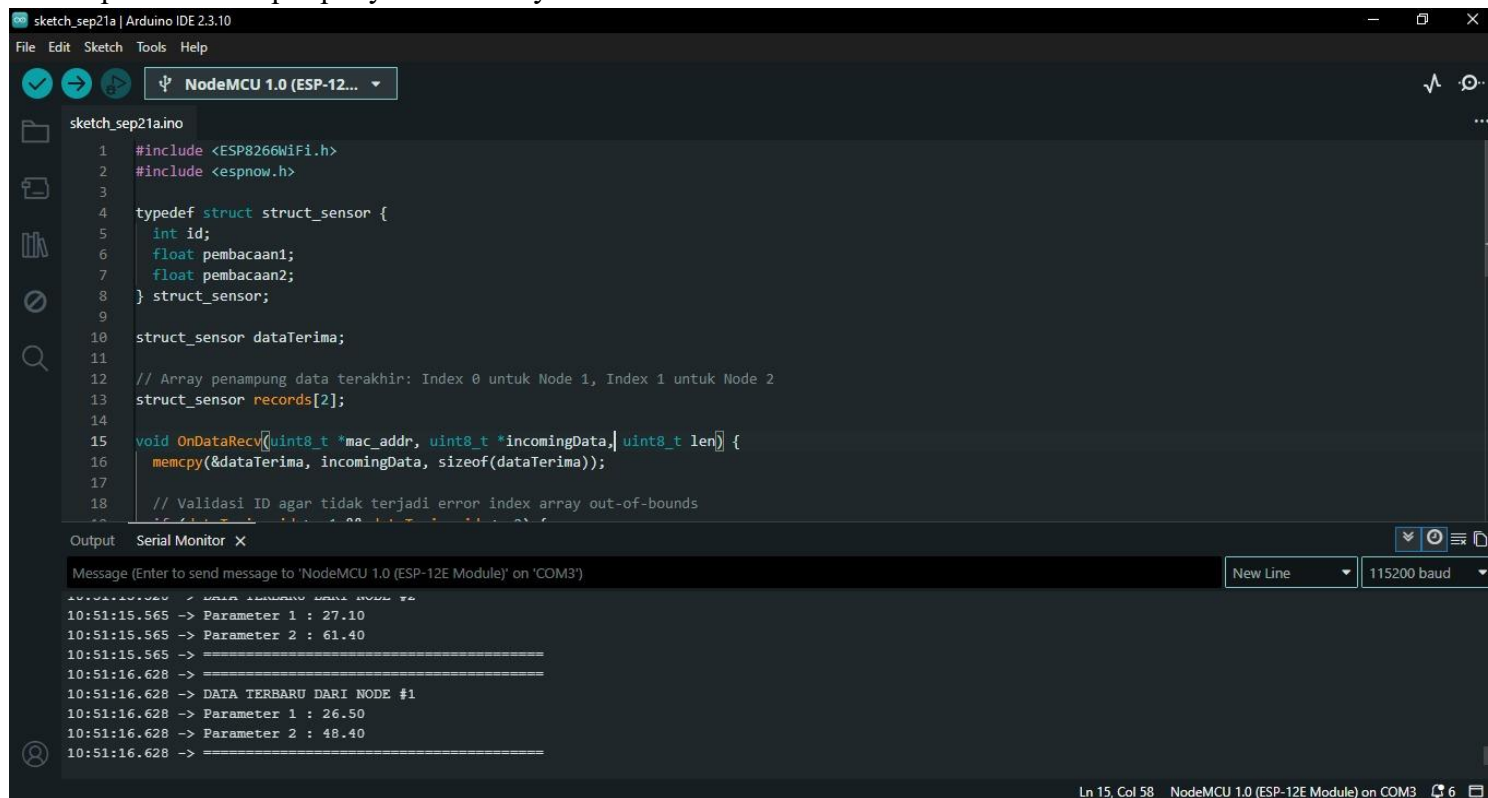
```

10:48:09.631 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:12.637 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:15.678 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:18.611 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:21.645 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:24.641 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:27.646 -> Pengiriman Node #2 -> Sukses Diterima Hub

```

Sekarang gentian saya yang mengirim ke teman saya dengan cara memasukan mac adres teman saya ke target kirim dan akhirnya berhasil

Ini tampilan dari output punya teman saya:



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading code, along with a dropdown menu for the board type, currently set to 'NodeMCU 1.0 (ESP-12...)'. The main editor displays a sketch named 'sketch_sep21a.ino' with the following code:

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
```

The bottom panel shows the 'Serial Monitor' window. It has a message input field and buttons for 'New Line' and '115200 baud'. The output shows the following data:

```
10:51:15.565 -> DATA TERBARU DARI NODE #1
10:51:15.565 -> Parameter 1 : 27.10
10:51:15.565 -> Parameter 2 : 61.40
10:51:15.565 -> =====
10:51:16.628 -> =====
10:51:16.628 -> DATA TERBARU DARI NODE #1
10:51:16.628 -> Parameter 1 : 26.50
10:51:16.628 -> Parameter 2 : 48.40
10:51:16.628 -> =====
```

The status bar at the bottom indicates 'Ln 15, Col 58 NodeMCU 1.0 (ESP-12E Module) on COM3'.

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor **DHT11** / **DHT22** pada masing-masing board NodeMCU:
 - Pin **VCC** sensor ke pin **3V3 (3.3V)** NodeMCU.
 - Pin **GND** sensor ke pin **GND** NodeMCU.
 - Pin **DATA / OUT** sensor ke pin **D4 (GPIO 2)** NodeMCU.

Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke **seluruh board** dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.

```
#include <painlessMesh.h>
```

```
#include <DHT.h>
```

```
#include <ArduinoJson.h>
```

```
#define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
```

```
#define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
```

```
#define MESH_PORT 5555 // Port komunikasi TCP Mesh
```

```
// Konfigurasi pin dan tipe sensor DHT
```

```
#define DHTPIN 2 // Pin D4 (GPIO 2)
```

```
#define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
// Identitas unik node untuk mempermudah identifikasi teks
```

```
const char* nodeName = "Node-1";
```

```
Scheduler userScheduler;
```

```
painlessMesh mesh;
```

```
// Prototipe fungsi pengiriman pesan berkala
```

```
void sendMessage();
```

```
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
```

```
void sendMessage() {
```

```
    // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
```

```
    float suhu = dht.readTemperature();
```

```
    float kelembapan = dht.readHumidity();
```

```
    // Validasi pembacaan sensor
```

```

if (isnan(suhu) || isnan(kelembapan)) {
    Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
    return;
}

// Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
StaticJsonDocument<200> doc;
doc["node"] = nodeName;
doc["chipId"] = mesh.getNodeId();
doc["suhu"] = suhu;
doc["kelembapan"] = kelembapan;

String msg;
serializeJson(doc, msg);

// Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
mesh.sendBroadcast(msg);

Serial.print("[KIRIM MESH] ");
Serial.println(msg);
}

// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
void receivedCallback(uint32_t from, String &msg) {
    // Parsing muatan teks JSON yang diterima
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, msg);

    if (!error) {
        const char* sender = doc["node"];
        uint32_t chipId = doc["chipId"];
        float suhu = doc["suhu"];
        float kelembapan = doc["kelembapan"];

        Serial.println("=====");
        Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
        Serial.printf("Suhu : %.2f °C\n", suhu);
        Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
        Serial.println("=====");
    } else {
        Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
    }
}

```

```

// Callback otomatis saat ada perangkat baru bergabung ke mesh
void newConnectionCallback(uint32_t nodeId) {
    Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
}

// Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
void changedConnectionCallback() {
    Serial.println("--> Topologi rantai mesh telah diperbarui");
}

void setup() {
    Serial.begin(115200);

    // Inisialisasi sensor DHT
    dht.begin();

    // Atur level log debugging (hanya tampilkan error dan status startup)
    mesh.setDebugMsgTypes(ERROR | STARTUP);

    // Inisialisasi jaringan mesh
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

    // Daftarkan event callback
    mesh.onReceive(&receivedCallback);
    mesh.onNewConnection(&newConnectionCallback);
    mesh.onChangedConnections(&changedConnectionCallback);

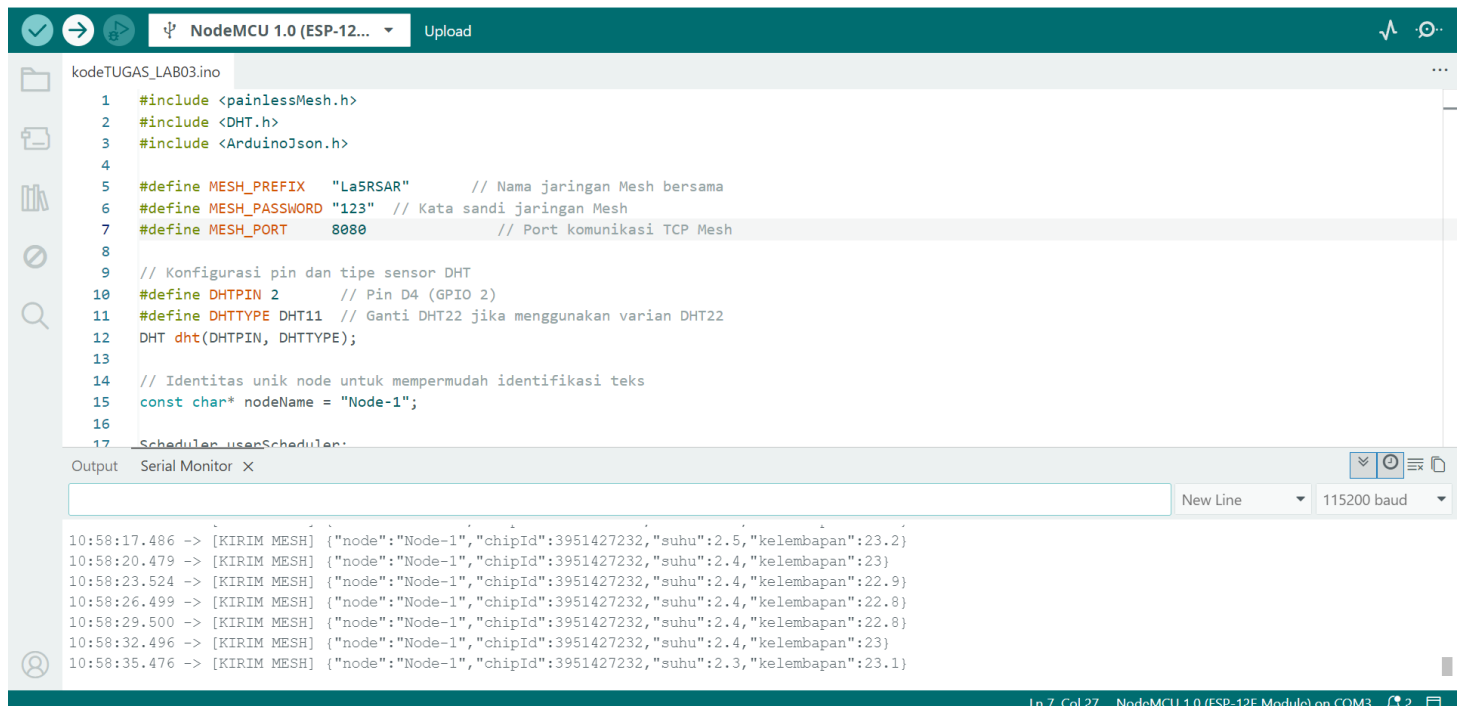
    // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
    userScheduler.addTask(taskSendMessage);
    taskSendMessage.enable();

    Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
}

void loop() {
    // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
    mesh.update();
}

```

Contoh gambar outputan



```
kodeTUGAS_LAB03.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "La5RSAR" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8080 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 #include <TaskScheduler.h>
18 TaskScheduler userScheduler;
```

Output Serial Monitor X

10:58:17.486 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.5,"kelembapan":23.2}
10:58:20.479 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.4,"kelembapan":23}
10:58:23.524 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.4,"kelembapan":22.9}
10:58:26.499 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.4,"kelembapan":22.8}
10:58:29.500 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.4,"kelembapan":22.8}
10:58:32.496 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.4,"kelembapan":23}
10:58:35.476 -> [KIRIM MESH] {"node":"Node-1","chipId":3951427232,"suhu":2.3,"kelembapan":23.1}

Penjelasan Kode Khusus painlessMesh & Integrasi JSON:

- `dht.readTemperature()` & `dht.readHumidity()` : Membaca nilai fisik suhu (° C) dan kelembapan () dari sensor DHT11/DHT22.
- `serializeJson(doc, msg)` : Mengubah struktur objek JSON menjadi deretan teks string ringkas siap siar.
- `deserializeJson(doc, msg)` : Membedah string teks JSON yang masuk kembali menjadi variabel numerik terpisah.
- `painlessMesh mesh;` : Menginisialisasi objek kontrol jaringan mesh yang menangani pembentukan soft-AP dan koneksi STA secara simultan di latar belakang.
- `mesh.getNodeId()` : Menghasilkan ID unik numerik berbasis MAC address bawaan chip ESP8266.
- `mesh.sendBroadcast(msg)` : Menyiarkan paket data teks ke seluruh node di dalam jaringan mesh. Paket akan diteruskan (*relayed*) otomatis oleh node perantara jika node target berada di luar jangkauan radio langsung.
- `TaskScheduler (userScheduler)` : Pengganti fungsi `delay()`. Di dalam jaringan mesh, penggunaan `delay()` dilarang keras karena akan membekukan tugas latar belakang perutean paket (*routing packet task*), yang dapat menyebabkan runtuhnya topologi mesh.
- `mesh.update()` : Menjalankan pemeliharaan koneksi, sinkronisasi jam internal mesh, dan pertukaran tabel rute secara kontinu di dalam siklus `loop()`

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Berdasarkan eksperimen pemutusan simpul, ketika seluruh NodeMCU dalam kondisi aktif, Controller menunjukkan status pengiriman "Berhasil Diterima". Setelah kabel USB NodeMCU 2 dicabut, pengiriman ke alamat MAC NodeMCU 2 menunjukkan status "Gagal (Tidak Terjangkau)", sedangkan NodeMCU 3 yang masih aktif tetap dapat menerima paket. Hal ini menunjukkan bahwa ESP-NOW memiliki mekanisme deteksi keberhasilan transmisi pada lapisan Data Link (Layer 2) melalui komunikasi berbasis alamat MAC dan respons pada tingkat MAC. Oleh karena itu, Controller dapat mengetahui bahwa perangkat tujuan tidak lagi terjangkau tanpa membutuhkan koneksi TCP. Kegagalan tersebut terjadi karena NodeMCU 2 yang telah kehilangan daya tidak dapat menerima frame maupun memberikan respons, sehingga `callback OnDataSent()` melaporkan kegagalan transmisi.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan `painlessMesh` pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:
 - i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
 - ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Pada eksperimen dinamika topologi, salah satu Node dimatikan dengan menekan tombol RST atau mencabut sumber dayanya. Setelah beberapa saat, node lain yang masih aktif mendeteksi perubahan topologi melalui `changedConnectionCallback()`. Waktu yang dibutuhkan untuk mendeteksi node yang keluar adalah sebesar ... **detik** berdasarkan hasil pengamatan. Selanjutnya, ketika sumber daya node tersebut disambungkan kembali, node secara otomatis mencari dan bergabung kembali ke jaringan `painlessMesh` tanpa konfigurasi ulang secara manual. Waktu yang dibutuhkan hingga node kembali terdaftar dalam topologi mesh adalah ... **detik**. Eksperimen ini menunjukkan kemampuan *self-healing* pada jaringan mesh, yaitu kemampuan jaringan untuk menyesuaikan topologi ketika terdapat node yang terputus maupun kembali aktif.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):**
Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

- Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL]` Telemetri dari Node <chipId> aman.
- Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for checking, running, and uploading code. The main editor window displays the code for `kodeTUGAS_LAB03.ino`, which is a NodeMCU 1.0 (ESP-12E) sketch. The code includes headers for `painlessMesh`, `DHT`, and `ArduinoJson`. It defines mesh parameters like `MESH_PREFIX`, `MESH_PASSWORD`, and `MESH_PORT`. It also configures the DHT sensor pin and type. The `receivedCallback` function is partially visible, showing the start of a JSON message handling routine. The Serial Monitor window at the bottom shows the output of the program, displaying a series of JSON messages from a node with chip ID 3951427232. The messages include temperature and humidity data, such as `{\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.6}`.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Lab5RSAR" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8080 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Tipe sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 // Scheduler userScheduler
```

Serial Monitor Output:

```
11:27:13.906 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.6}
11:27:16.895 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.8}
11:27:19.895 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 51.2}
11:27:22.917 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.9}
11:27:25.929 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.9}
11:27:28.915 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.8}
11:27:31.935 -> [KIRIM MESH] {\"node\": \"Node-1\", \"chipId\": 3951427232, \"suhu\": 31.2, \"kelembapan\": 50.7}
```

5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis *painlessMesh*

Kembangkan jaringan mesh pada Praktikum 4 menjadi sistem *monitoring* lingkungan dan kendali otomasi terdistribusi tanpa perantara router. Setiap kelompok membagi peran perangkat sebagai berikut:

- **Tujuan:** Menguasai transmisi paket JSON terdistribusi melintasi jaringan *Ad-Hoc Mesh Multi-Hop* dan mengevaluasi *rule engine* otomasi aktuator LED di node konsentrator.
- **Tingkat Kesulitan:** Lanjut (Kolaboratif Kelompok)
- **Instruksi Perangkaian:**
 - i. **Node 1 (Stasiun Lingkungan):** Hubungkan pin DATA sensor **DHT11/DHT22** ke pin **D4 (GPIO 2)** NodeMCU 1 (VCC ke 3V3, GND ke GND).
 - ii. **Node 2 (Stasiun Cahaya):** Rangkai sensor **LDR** dan **Resistor 10k Ohm** sebagai pembagi tegangan ke pin **A0** NodeMCU 2.
 - iii. **Node 3 (Hub Kontrol Aktuator):**
 - Hubungkan kaki Anoda (kaki panjang) **LED** ke pin **D6 (GPIO 12)** NodeMCU 3.
 - Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke jalur **GND** (*rangkaian LED aktif-HIGH seperti pada Modul 3 dan Modul 4*).
- **Instruksi Pemrograman (Front-End & Back-End Mesh):**
 - i. **Node 1 (Sensor DHT):**
 - Modifikasi fungsi `sendMessage()` agar membaca suhu dan kelembapan riil menggunakan pustaka `DHT.h` setiap 3 detik.
 - Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:
`{"tipe":"suhu_node","suhu":29.4,"kelembapan":65.0}`
 - ii. **Node 2 (Sensor LDR):**
 - Modifikasi fungsi `sendMessage()` agar membaca nilai ADC analog LDR setiap 3 detik.
 - Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:
`{"tipe":"cahaya_node","adc":420}`
 - iii. **Node 3 (Hub Aktuator Cerdas):**
 - Nonaktifkan `taskSendMessage` pada Node 3 (node ini bertindak pasif sebagai penerima dan pengambil keputusan).
 - Pada fungsi `receivedCallback(uint32_t from, String &msg)`:
 - Gunakan pustaka `ArduinoJson` (`JsonDocument / StaticJsonDocument<200>`) untuk membedah string JSON masuk.
 - Evaluasi kondisi *Rule Engine*:
 - **JIKA** data suhu dari Node 1 bernilai **di atas 31.0 Celcius** ATAU nilai ADC cahaya dari Node 2 bernilai **di bawah 300** (kondisi gelap), **nyalakan LED** (`digitalWrite(ledPin, HIGH)`).
 - Jika kedua parameter kembali ke kondisi normal, **matikan LED** (`digitalWrite(ledPin, LOW)`).
 - iv. **Eksperimen Ketahanan Jaringan (*Multi-Hop* dan *Self-Healing*):**
 - Jauhkan Node 1 dan Node 3 hingga berada di luar jangkauan radio langsung masing-masing.
 - Tempatkan Node 2 di posisi tengah antara Node 1 dan Node 3.

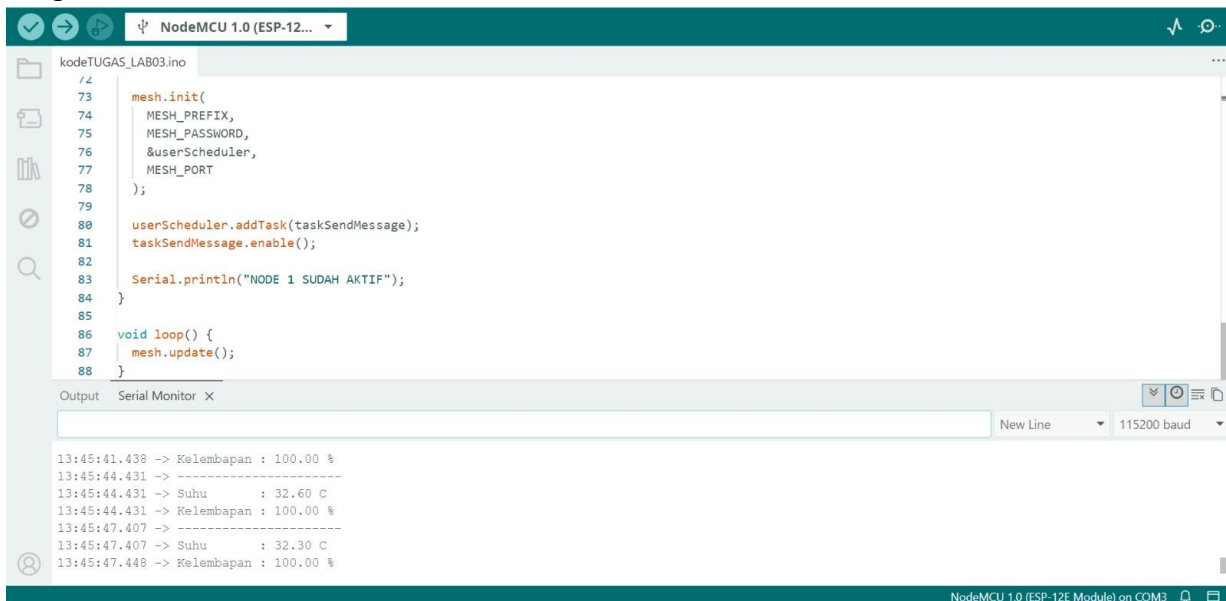
- Amati pada Serial Monitor apakah paket telemetri dari Node 1 tetap dapat diterima oleh Node 3 melalui relai perantara (*multi-hop relay*) Node 2.
- Matikan daya Node 2 (simulasi node putus/gagal), lalu amati perubahan status pada Serial Monitor Node 3 melalui *callback* `changedConnectionCallback()`.
- **Kriteria Keberhasilan:**
 - Ketiga node berhasil membentuk jaringan rantai mesh otonom tanpa menggunakan router Wi-Fi eksternal.
 - Data sensor riil dari Node 1 dan Node 2 berhasil diterima dan di-*parsing* dengan benar oleh Node 3 menggunakan format JSON.
 - Logika *Rule Engine* pada Node 3 berhasil memicu aktuator LED fisik sesuai kondisi ambang batas (*threshold*).
 - Mekanisme *Multi-Hop Relay* dan *Self-Healing* teramati dengan jelas melalui log Serial Monitor.

[PENTING!] Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan **commit dan push ke repositori GitHub pribadi Anda masing-masing**.

Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

Hasil dari tugas

Outputan dari node1



The screenshot shows the Arduino IDE interface with the file 'kodeTUGAS_LAB03.ino' open. The code defines a mesh network and a task to send messages. The serial monitor shows the output of the program, which includes the status 'NODE 1 SUDAH AKTIF' and sensor data for humidity and temperature.

```
kodeTUGAS_LAB03.ino
12
73 mesh.init(
74   MESH_PREFIX,
75   MESH_PASSWORD,
76   &userScheduler,
77   MESH_PORT
78 );
79
80 userScheduler.addTask(taskSendMessage);
81 taskSendMessage.enable();
82
83 Serial.println("NODE 1 SUDAH AKTIF");
84 }
85
86 void loop() {
87   mesh.update();
88 }
```

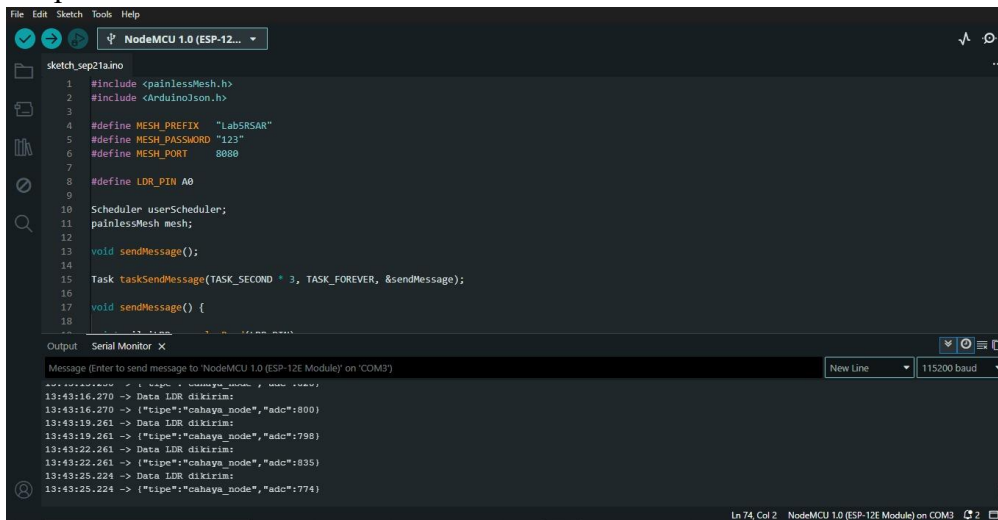
Output Serial Monitor X

13:45:41.438 -> Kelembapan : 100.00 %
13:45:44.431 -> -----
13:45:44.431 -> Suhu : 32.60 C
13:45:44.431 -> Kelembapan : 100.00 %
13:45:47.407 -> -----
13:45:47.407 -> Suhu : 32.30 C
13:45:47.448 -> Kelembapan : 100.00 %

NodeMCU 1.0 (ESP-12E Module) on COM3

Node satu menampilkan sensor dht 22

Output dari node 2



The screenshot shows the Arduino IDE interface with the file 'sketch_sep21a.ino' open. The code defines a mesh network and a task to send messages. The serial monitor shows the output of the program, which includes the status 'NODE 2 SUDAH AKTIF' and sensor data for LDR and temperature.

```
sketch_sep21a.ino
1 #include <painlessMesh.h>
2 #include <ArduinoJson.h>
3
4 #define MESH_PREFIX "LabSRASAR"
5 #define MESH_PASSWORD "123"
6 #define MESH_PORT 8080
7
8 #define LDR_PIN A0
9
10 Scheduler userScheduler;
11 painlessMesh mesh;
12
13 void sendMessage();
14
15 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
16
17 void sendMessage() {
18   // ...
19 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

13:43:16.270 -> Data LDR dikirim:
13:43:16.270 -> ("type":"cahaya_node","adc":800)
13:43:19.261 -> Data LDR dikirim:
13:43:19.261 -> ("type":"cahaya_node","adc":798)
13:43:22.261 -> Data LDR dikirim:
13:43:22.261 -> ("type":"cahaya_node","adc":835)
13:43:25.224 -> Data LDR dikirim:
13:43:25.224 -> ("type":"cahaya_node","adc":774)

Ln 74, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3

Disini node 2 menampilkan outputan dari sensor ldr dan naantinya akan di tampillkan oleh node 3

Outputan node 3 final

```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
Blinkino
1 #include "painlessMesh.h"
2 #include <ArduinoJson.h>
3
4 #define MESH_PREFIX "Lab5Medan"
5 #define MESH_PASSWORD "Medan1234"
6 #define MESH_PORT 8080
7
8 painlessMesh mesh;
9
10 Scheduler usersScheduler;
11
12 const int ledPin = D6;
13
14 // Menyimpan data terakhir dari Node 1 dan Node 2
15 float suhu = 0.0;
16 int adcCahaya = 1023;
17
18 // Status apakah data sudah pernah diterima
19 bool suhuDiterima = false;
20 bool cahayaDiterima = false;
21
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
Pesan JSON: {"tipe":"cahaya_node","adc":3}
Sumber: Node 2 - Sensor LDR
ADC Cahaya : 3
----- RULE ENGINE -----
Suhu > 31 C : TIDAK
Cahaya < 300 : YA
KEPUTUSAN: LED ON
Alasan: Suhu tinggi ATAU kondisi gelap
=====
```

Disinni finalnya node 3 menampilkan semua hasil dari node 1 dan node 2 jika sensor Cahaya dan sensor suhu melebihi ambang batass maka lampu led dari node satu akan hidup

Bukti foto dari rangkaian

